ANCHOR CBS · INSA CYBERSECURITY AUDIT

Document: README.md · Generated 11 November 2026

Security Audit — Runbook and Evidence Guide

System: CBS General — Web-Based SACCO Core Banking / ERP Back Office
Document ID: CBS-SEC-RB-2026-001
Version: 1.0
Date: 11 November 2026
Classification: Internal / Confidential

This guide explains how to run the security checks, configure the environment, harden a production deployment, and reproduce the evidence cited in:

- `security/COMPLIANCE_MATRIX.md`
- `security/VULNERABILITY_REPORT.md`
- `security/SRS.md`

> **Scope reminder:** this is a web back office. It has no POS terminal, invoice machine, card reader, firmware, TPM, secure boot, OTA update, or fiscal printer. Device-only INSA checklist items are **Not Applicable**; see the matrix.

---

1. Deliverables

File	Purpose
`security/COMPLIANCE_MATRIX.md`	52-row INSA OF/AEAD/001 checklist with status, evidence, and remediation.
`security/VULNERABILITY_REPORT.md`	19 findings (9 fixed, 5 partial, 5 open), CVSS-style ratings, evidence, test results, roadmap, and the 2026-11-11 remediation-update table.
`security/SRS.md`	Requirements specification for the implemented SACCO platform.
`security/evidence/security-test-results.md`	Live test transcript and build/regression results for the hardening batch.
`security/README.md`	This runbook: checks, environment, hardening, evidence reproduction.

---

2. Environment Variables

Set these in the process environment, a `.env` file loaded by your launcher, or a secret manager. **Do not commit real values.** `.env` and `.env.*` are gitignored; `backend/.env.example` contains placeholders only.

Variable	Required in production	Default	Purpose
`JWT_SECRET`	**Yes**	Dev fallback in `application.yml`	HMAC key for JWT signing. Use ≥ 32 random bytes; rotate on compromise (invalidates all sessions). `PRODUCTION=true` rejects the demo default.

Variable	Required in production	Default	Purpose
`JWT_EXPIRATION_MS`	Recommended	`1800000` (30 min)	Access-token lifetime. Keep at 30 minutes or lower.
`DB_URL`	Recommended	`jdbc:postgresql://localhost:5432/cbs`	PostgreSQL JDBC URL. Use `sslmode=verify-full` in production.
`DB_USERNAME`	Recommended	`cbs_app`	Runtime database user. Grant only required DML; use a separate migration user.
`DB_PASSWORD`	**Yes**	Dev fallback	Database password.
`PRODUCTION`	**Yes**	`false`	Enables the fail-fast credential guard. Set `true` in production; `SEED_PASSWORD=demo123` or the demo JWT secret then aborts startup.
`SEED_PASSWORD`	**Yes in production**	`demo123`	Password used only when `DataSeeder` creates missing demo users. Set a strong, unique value in production.
`REQUIRE_MFA`	**Yes**	`false`	Enables SMS OTP for ADMIN/MANAGER. Set `true` in production.
`MAX_FAILED_ATTEMPTS`	Recommended	`5`	Failed logins before lockout.
`LOCK_MINUTES`	Recommended	`15`	Lockout duration.
`REQUIRE_HTTPS`	**Yes**	`false`	Redirects requests to HTTPS and enables secure-channel enforcement. Set `true` behind the TLS proxy.
`CORS_ORIGINS`	**Yes**	`http://localhost:5173,http://127.0.0.1:5173`	Comma-separated exact browser origins. Never use `*`.
`AFRO_SMS_ENABLED`	If MFA/SMS is used	`false`	Enables the AfroMessage gateway.
`AFRO_SMS_API_KEY`	If SMS is used	empty	Server-side bearer token. Never expose to the browser.
`AFRO_SMS_FROM`	No	empty	Optional gateway `from` value.
`AFRO_SMS_SENDER`	No	empty	Optional sender name.

Example `.env` for a local run (replace values):

```
[redacted]
```

```
SEED_PASSWORD=demo123
REQUIRE_MFA=false
REQUIRE_HTTPS=false
CORS_ORIGINS=http://localhost:5173
AFRO_SMS_ENABLED=false
AFRO_SMS_API_KEY=
AFRO_SMS_FROM=
AFRO_SMS_SENDER=
```

For a production-like run, use `PRODUCTION=true`, a strong `SEED_PASSWORD`, and a random `JWT_SECRET`. The application will fail startup if the demo seed password or demo JWT secret is still in use.

Generate a secret:

```
openssl rand -base64 48
```

---

3. Running the System Locally

3.1 Prerequisites

- Java 21, Maven
- Node.js 20+ and npm
- PostgreSQL 14+ with the `cbs` database and `cbs_app` role
- Optional: an AfroMessage account for live SMS/OTP

3.2 Start the backend

```
cd backend
set -a && . ./.env && set +a
mvn spring-boot:run
```

The API listens on `http://localhost:8080`. Flyway applies migrations V1–V29 on startup.

3.3 Start the front end

```
npm install
npm run dev
```

Vite serves `http://localhost:5173` and proxies `/api` to `:8080` (see `vite.config.ts`).

3.4 Local run with MFA

```
cd backend
export PRODUCTION=false             # demo mode; use true + strong secrets in production
export SEED_PASSWORD=demo123        # demo only
export REQUIRE_HTTPS=false          # keep false for plain-HTTP local tests
export REQUIRE_MFA=true             # only if the SMS gateway is configured
export CORS_ORIGINS=http://localhost:5173
mvn spring-boot:run
```

> `REQUIRE_HTTPS=true` without a TLS terminator will redirect local HTTP requests. Use it only behind nginx/ALB with forwarded headers. With `PRODUCTION=true`, startup fails if `SEED_PASSWORD=demo123` or the demo JWT secret is used.

---

4. Security Checks

Set a base URL and test credentials. Use dedicated test accounts, not production users.

```
export BASE=http://localhost:8080
export TELLER_USER=teller
export TELLER_PASS='<teller-password>'
export AUDITOR_USER=auditor
export AUDITOR_PASS='<auditor-password>'
```

4.1 Unauthenticated access

```
curl -s -o /dev/null -w 'members=%{http_code}\n' "$BASE/api/members"
curl -s -o /dev/null -w 'system=%{http_code}\n'  "$BASE/api/system/health"
```

Expected: `401` for both.

4.2 Password authentication

```
# Missing password
curl -s -o /dev/null -w '%{http_code}\n' -X POST "$BASE/api/auth/login" \
  -H 'Content-Type: application/json' -d '{"username":"admin"}'

# Wrong password
curl -s -o /dev/null -w '%{http_code}\n' -X POST "$BASE/api/auth/login" \
  -H 'Content-Type: application/json' -d '{"username":"teller","password":"wrong"}'
```

Expected: `401` with `{"status":401,"error":"Invalid username or password"}`.

> Do not run repeated wrong-password tests against a real account; five failures lock it for 15 minutes.

4.3 Obtain tokens

```
TOKEN=$(curl -s -X POST "$BASE/api/auth/login" \
  -H 'Content-Type: application/json' \
  -d "{\"username\":\"$TELLER_USER\",\"password\":\"$TELLER_PASS\"}" \
  | python3 -c 'import sys,json; print(json.load(sys.stdin).get("token",""))')

test -n "$TOKEN" && echo "token acquired"

AUDITOR_TOKEN=$(curl -s -X POST "$BASE/api/auth/login" \
  -H 'Content-Type: application/json' \
  -d "{\"username\":\"$AUDITOR_USER\",\"password\":\"$AUDITOR_PASS\"}" \
  | python3 -c 'import sys,json; print(json.load(sys.stdin).get("token",""))')
```

4.4 RBAC matrix

```
for ep in /api/auth/users /api/members /api/audit-trail /api/system/backups; do
  printf '%-24s ' "$ep"
  curl -s -o /dev/null -w '%{http_code}\n' \
    -H "Authorization: Bearer $TOKEN" "$BASE$ep"
done
```

Expected for TELLER: `403`, `200`, `403`, `403`.

```
# AUDITOR can read audit but cannot write
curl -s -o /dev/null -w 'audit=%{http_code}\n' \
  -H "Authorization: Bearer $AUDITOR_TOKEN" "$BASE/api/audit-trail"
curl -s -o /dev/null -w 'write=%{http_code}\n' -X POST \
  -H "Authorization: Bearer $AUDITOR_TOKEN" -H 'Content-Type: application/json' \
  -d '{}' "$BASE/api/members"
```

Expected: `audit=200`, `write=403`.

4.5 Fine-grained authorization

```
# A TELLER must not reach GL or compliance writes.
curl -s -o /dev/null -w 'teller-gl-rules=%{http_code}\n' -X POST \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{}' "$BASE/api/gl-rules"

curl -s -o /dev/null -w 'teller-compliance=%{http_code}\n' -X POST \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{}' "$BASE/api/compliance/policy"
```

Expected: `403` for both. A TELLER may still post savings transactions (`400` validation is expected, because the role is allowed).

For an ACCOUNTANT token, `/api/gl-rules` is allowed (`400` validation) while `/api/compliance/policy` returns `403`. This is the current **V-16 Partial** boundary: sensitive POSTs are restricted, but PUT/PATCH, `/api/sms/**`, `/api/accounting/**`, and some other writes remain broad.

4.6 Security headers and HSTS

```
curl -s -D - -o /dev/null "$BASE/api/members" \
  | grep -Ei 'strict-transport|content-security|x-frame|x-content|referrer|permissions'

# Simulate a TLS-terminated request (the app trusts forwarded headers)
curl -s -D - -o /dev/null -H 'X-Forwarded-Proto: https' "$BASE/api/members" \
  | grep -i strict-transport
```

Expected headers: `Content-Security-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, `Permissions-Policy`; HSTS with `max-age=31536000; includeSubDomains` when secure.

4.7 CORS allowlist

```
# Allowed origin
curl -s -D - -o /dev/null -X OPTIONS \
  -H 'Origin: http://localhost:5173' \
  -H 'Access-Control-Request-Method: GET' \
  "$BASE/api/members" | grep -i 'access-control-allow-origin'
```

```
# Disallowed origin
curl -s -D - -o /dev/null -H 'Origin: https://evil.example' \
  "$BASE/api/members" | grep -i 'access-control-allow-origin' || echo "no ACAO (expected)"
```

Expected: allowed origin returns `Access-Control-Allow-Origin`; disallowed origin returns none.

4.8 Input validation and injection

```
# Negative amount (teller token)
curl -s -o /dev/null -w '%{http_code}\n' -X POST "$BASE/api/savings/transactions" \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"memberId":1,"productId":1,"type":"Deposit","amount":-500}'

# SQL injection in login
curl -s -o /dev/null -w '%{http_code}\n' -X POST "$BASE/api/auth/login" \
  -H 'Content-Type: application/json' \
  -d '{"username":"'"'"' OR 1=1--","password":"x"}'
```

Expected: `400` ("Amount must be a positive number") and `401` ("Invalid username or password").

4.9 Audit trail integrity

```
curl -s -H "Authorization: Bearer $AUDITOR_TOKEN" "$BASE/api/audit-trail/verify"
```

Expected: `{"valid":true,"checked":<n>}`. The count grows with every audited change; it is not a fixed value.

```
# Append-only triggers
set -a && . backend/.env && set +a
PGPASSWORD="$DB_PASSWORD" psql -h localhost -U "${DB_USERNAME:-cbs_app}" -d "${DB_NAME:-cbs}" \
  -tAc "select c.relname||' -> '||t.tgname
        from pg_trigger t join pg_class c on c.oid=t.tgrelid
        where not t.tgisinternal and c.relname in ('audit_log','audit_trail','sms_log')
        order by 1;"
```

Expected:

```
audit_log -> audit_log_append_only
audit_trail -> audit_trail_append_only
sms_log -> sms_log_append_only
```

Do **not** run manual `UPDATE`/`DELETE` against these tables in production; the trigger raises an exception by design. `audit_log` is append-only but not hash-chained; prefer `audit_trail` for evidence.

4.10 JWT lifetime

```
python3 - "$TOKEN" <<'PY'
import sys, base64, json
t = sys.argv[1].split('.')[1]
t += '=' * (-len(t) % 4)
p = json.loads(base64.urlsafe_b64decode(t))
print({"sub": p.get("sub"), "role": p.get("role"), "ttl_seconds": p.get("exp") - p.get("iat")})
PY
```

Expected: `ttl_seconds` = `1800`.

4.11 Dependency scan

```
npm audit --omit=dev
npm ls @ant-design/pro-components @ant-design/pro-layout
```

Expected: `found 0 vulnerabilities`; the `npm ls` output is empty.

Backend dependency scan (recommended, not run in this audit):

```
cd backend
mvn org.owasp:dependency-check-maven:check
```

4.12 Error handling

```
curl -s -X POST "$BASE/api/savings/transactions" \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"memberId":1,"productId":1,"type":"Deposit"}'
```

Expected: a generic JSON `400` / `500` -class error with no stack trace, class names, or SQL. `server.error.include-stacktrace: never` and `include-message: never` are set.

4.13 MFA and lockout (use deliberately)

These tests have side effects. Use a dedicated test account and a test phone number.

```
# With REQUIRE_MFA=true and a MANAGER account:
curl -s -X POST "$BASE/api/auth/login" -H 'Content-Type: application/json' \
  -d '{"username":"manager","password":"<password>"}'
```

Expected: `202 {"mfaRequired":true,...}` and an SMS code. Do not repeat to avoid SMS cost.

Lockout: five wrong passwords return `401`; the sixth returns `423`. Wait 15 minutes or clear the test account's `locked_until` / `failed_attempts` in a non-production database.

4.14 Production credential guard

```
cd backend
PRODUCTION=true SEED_PASSWORD=demo123 JWT_SECRET='cbs-general-demo-secret-key-change-me-in-production-0123456789abcdef' \
  mvn spring-boot:run
```

Expected: startup aborts with `PRODUCTION=true requires a non-default SEED_PASSWORD` (or the JWT-secret message). Repeat with `SEED_PASSWORD` set to a strong value and a random `JWT_SECRET`; startup should proceed. The guard only rejects the literal `demo123` and the demo JWT prefix, so also enforce length/entropy in the deployment manifest.

---

5. Production Hardening Checklist

#	Control	Requirement	Evidence to retain
1	TLS 1.3	Deploy `deploy/nginx-cbs.conf.template` (or equivalent ALB policy) with TLS 1.3 only, AEAD ciphers, and automated certificates.	`openssl s_client -tls1_3`, SSL Labs A+ report, deployed proxy config.

#	Control	Requirement	Evidence to retain
2	HTTPS enforcement	Redirect all HTTP to HTTPS; set `REQUIRE_HTTPS=true`; HSTS preload after validation.	Redirect test, HSTS header, preload status.
3	mTLS	Client certificates for service-to-service and any POS/ERP connector.	Truststore config, negative handshake test.
4	Secret manager	JWT/DB/SMS secrets in Vault/AWS Secrets Manager/KMS; no `.env` on hosts; rotate on schedule and on compromise.	Secret inventory, rotation log, access policy.
5	Remove dev defaults	Set `PRODUCTION=true`; provide strong `JWT_SECRET`/`DB_PASSWORD`; remove `application.yml` fallbacks or fail startup.	Startup failure test, config diff.
6	Demo accounts	Set a strong `SEED_PASSWORD`; rotate any existing `demo123` hashes; gate `DataSeeder` behind a demo profile; force first-login change.	User inventory, credential test, seed-profile config.
7	MFA	`REQUIRE_MFA=true`; enrol all ADMIN/MANAGER users; document recovery and break-glass.	Enrolment records, 202/200 tests, recovery procedure.
8	CORS	Exact production origins only; no wildcard; credentials disabled.	`CORS_ORIGINS`, preflight test.
9	Authorization	Per-endpoint role matrix; no TELLER/CREDIT_OFFICER writes to GL/compliance/admin; extend to PUT/PATCH, SMS, and accounting; AUDITOR read-only.	Role × endpoint test report (V-16 residual closure).
10	Encryption at rest	Encrypted volumes + PostgreSQL TDE; AES-256-GCM or tokenization for Fayda ID/phone; encrypted backups.	KMS/volume config, schema/cipher evidence, restore test.
11	Database least privilege	Runtime user with DML only; separate migration user; no superuser; TLS to DB.	`\du`, grants, `pg_hba.conf`, connection string.
12	Audit export	Stream `audit_trail` to WORM/SIEM; `audit_log` is now append-only but not hash-chained — retire it or reconcile; alert on `verify` failure and trigger errors; retain per policy.	SIEM pipeline, retention policy, alert runbook.
13	Backups	Daily full + WAL archiving; encrypted; off-site; quarterly restore test; RPO/RTO documented.	Backup schedule, restore report, encryption evidence.
14	Monitoring	Alert on failed-login spikes, lockouts, 403 spikes, 5xx, SMS failures, audit-chain failures, disk/DB health.	Dashboards, alert rules, on-call runbook.
15	Patching	Monthly OS/DB/JDK patches; `npm audit` and Maven Dependency-Check in CI; container image scanning.	Scan reports, patch log.
16	Pen test	Independent authenticated web/API test before go-live and annually; remediate and retest.	Signed report, retest letter.
17	Retention	Data classification, retention schedule, purge/anonymization, legal hold.	Approved policy, purge job logs.
18	Front-end hardening	CSP at the static host, no `localStorage` token, XSS sanitization, dependency pinning.	CSP header, code review, ZAP results.
19	Logging hygiene	No secrets/PII in logs; redact phone/Fayda; centralize with access control.	Log sample, pipeline config.
20	Change control	Security review for auth/RBAC/schema changes; migrations tested; rollback plan.	Change tickets, review records.

---

6. Reproducing the Evidence

Evidence in the report	How to reproduce
Unauthenticated `401`	§4.1.

Evidence in the report	How to reproduce
Password enforcement `401`	§4.2.
JWT `ttl_seconds=1800`	§4.10.
TELLER/MANAGER `403` on `/api/auth/users`; ADMIN `200`	§4.4 with each role.
AUDITOR read `200`, write `403`	§4.4.
TELLER finance/compliance write `403`; ACCOUNTANT GL `400`, compliance `403`	§4.5.
Negative amount `400`; SQLi `401`	§4.8.
Audit chain `valid:true` (`checked:58` in the evidence run)	§4.9.
Append-only triggers on `audit_trail`, `sms_log`, `audit_log`	§4.9 SQL query.
Live test transcript	`security/evidence/security-test-results.md`.
Security headers / HSTS	§4.6.
CORS allow/deny	§4.7.
Zero npm vulnerabilities (before: 3 high)	§4.11.
Production credential guard	Start with `PRODUCTION=true` and `SEED_PASSWORD=demo123`; code-inspected fail-fast in `SecurityBootstrapCheck` (not exercised here).
MFA `202` / teller `200`	§4.13 with `REQUIRE_MFA=true`; teller login via §4.3.
Lockout `401` ×5 then `423`	§4.13, dedicated account only.
Password policy `400` for `"abc"`	`POST /api/auth/users` as ADMIN with `{"username":"x","name":"X","role":"TELLER","password":"abc"}`.
No stack traces	§4.12 + `application.yml`.

Known test-harness caveat. `scripts/api-smoke.sh`, `BaseApiIT`, `SaccoApiIntegrationTest`, `ApiContractIT`, `PerformanceIT`, and `EdgeCasesIT` still use the legacy passwordless login payload. They will fail against the remediated backend and must be updated before being used as release evidence:

```
# Legacy (no longer valid)
-d '{"username":"admin"}'
-d '{"role":"MANAGER"}'

# Current contract
-d '{"username":"admin","password":"<password>"}'
```

Also note `src/context/AuthContext.tsx` `switchRole` posts only `{role}` and will return `401`; it needs re-authentication or removal.

---

7. Known Limitations

- **No POS/invoice hardware.** Device, firmware, secure-boot, OTA, TPM, JTAG/UART, fiscal-printer, and physical-security checklist items are N/A. Do not present this system as device-certified.
- **TLS template is not deployed.** `deploy/nginx-cbs.conf.template` provides TLS 1.3, HSTS, and HTTP→HTTPS, but the audited host still serves HTTP; mTLS is absent (V-10 Partial).
- **No verified encryption at rest.** Volume/TDE encryption and field-level encryption are open (V-11).
- **No independent penetration test or certification.** ISO 27001, ISO 15408, PCI-DSS, and fiscal certification are not held.
- **Demo mode still uses `demo123`.** `PRODUCTION=true` rejects the demo seed/JWT at startup, but `PRODUCTION=false` is the default, existing databases may retain old hashes, and `.env.example` is only a template (V-15 Partial).
- **Authorization remains partial** (V-16). Sensitive POSTs are restricted, but PUT/PATCH, `/api/sms/**`, `/api/accounting/**`, and some writes are still broad; do not rely on the UI role menu.
- **MFA is not fully operationalized.** SMS OTP works for ADMIN/MANAGER but `REQUIRE_MFA` is off by default; enrolment, reset, and recovery are manual (V-09/V-14 Partial).
- **`audit_log` is append-only but not hash-chained.** UPDATE/DELETE are blocked by V29; INSERT remains possible, and the table can diverge from `audit_trail` (V-19 Fixed with residual).
- **Audit trail count** (`checked:58` in the evidence run) is a point-in-time value and will change.

---

8. Incident Response Hooks

If audit-chain verification fails or append-only triggers report errors:

1. Preserve the database and application logs; do not attempt to "repair" the chain in place.
2. Isolate the API and database network paths.
3. Capture `GET /api/audit-trail/verify`, the first `brokenAt` ID, and database trigger/error logs.
4. Notify the SACCO security officer and, where required, INSA.
5. Restore from a known-good backup and re-establish the chain only under a documented forensic procedure.

End of runbook.